

Évaluation 1 :

Programmation Orientée Objet

1. Gestion de stocks - 15 pts -

Un fabricant de brioches décide d'informatiser sa gestion des stocks. Il écrit pour cela un programme en langage Python. Une partie de son travail consiste à développer une classe `Stock` dont la première version est la suivante :

```
class Stock:
    def __init__(self):
        self.qt_farine = 0 # quantite de farine initialisee a 0 g
        self.nb_oeufs = 0 # nombre d'oeufs (0 a l'initialisation)
        self.qt_beurre = 0 # quantite de beurre initialisee a 0 g
```

1. **Écrire** une méthode `ajouter_beurre(self, qt)` qui ajoute la quantité `qt` de beurre à un objet de la classe `Stock`.

On admet que l'on a écrit deux autres méthodes `ajouter_farine` et `ajouter_oeufs` qui ont des fonctionnements analogues.

2. **Écrire** une méthode `afficher(self)` qui affiche la quantité de farine, d'œufs et de beurre d'un objet de type `Stock`. L'exemple ci-dessous illustre l'exécution de cette méthode dans la console :

```
>>> mon_stock = Stock()
>>> mon_stock.afficher()
farine: 0
oeuf: 0
beurre: 0
>>> mon_stock.ajouter_beurre(560)
>>> mon_stock.afficher()
farine: 0
oeuf: 0
beurre: 560
```

3. Pour faire une brioche, il faut 350 g de farine, 175 g de beurre et 4 oeufs. **Écrire** une méthode `stock_suffisant_brioche(self)` qui renvoie un booléen : `VRAI` s'il y a assez d'ingrédients dans le stock pour faire une brioche et `FAUX` sinon.

4. On considère la méthode supplémentaire `produire(self)` de la classe `Stock` donnée par le code suivant :

```
def produire(self):
    res = 0
    while self.stock_suffisant_brioche():
        self.qt_beurre = self.qt_beurre - 175
        self.qt_farine = self.qt_farine - 350
        self.nb_oeufs = self.nb_oeufs - 4
        res = res + 1
    return res
```



On considère un stock défini par les instructions suivantes :

```
>>> mon_stock=Stock()  
>>> mon_stock.ajouter_beurre(1000)  
>>> mon_stock.ajouter_farine(1000)  
>>> mon_stock.ajouter_oeufs(10)
```

a. On exécute ensuite l'instruction

```
>>> mon_stock.produire()
```

5. Indiquer quelle valeur s'affiche dans la console ? **Préciser** ce que représente cette valeur ?

b. On exécute ensuite l'instruction

```
>>> mon_stock.afficher()
```

6. Indiquer quelle valeur s'affiche dans la console ?

7. L'industriel possède n lieux de production distincts et donc n stocks distincts. On suppose que ces stocks sont dans une liste dont chaque élément est un objet de type `Stock`. **Écrire** une fonction Python `nb_brioche(liste_stocks)` possédant pour unique paramètre la liste des stocks et renvoie le nombre total de brioches produites.

2. Positionnement d'antennes - 15 pts -

Une entreprise doit placer des antennes relais le long d'une rue rectiligne. Une antenne relais de portée (ou *rayon*) pp couvre toutes les maisons qui sont à une distance inférieure ou égale à pp de l'antenne.

Connaissant les positions des maisons dans la rue, l'objectif est de placer les antennes le long de la rue, pour que toutes les maisons soient couvertes, tout en minimisant le nombre d'antennes utilisées.

La rue est représentée par un axe, et les maisons sont représentées des points sur cet axe :

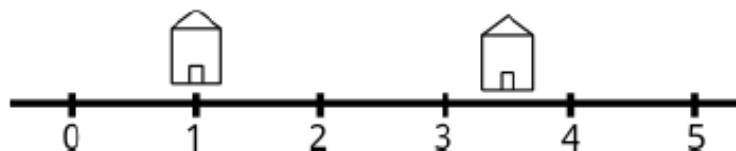


Figure 1. Deux maisons sur une rue, repérée par leur abscisse : 1 et 3,5



Les entités manipulées sont modélisées en utilisant la programmation orientée objet.

```
class Maison:
    def __init__(self, position):
        self._position = position
    def get_pos_maison(self):
        return self._position
class Antenne:
    def __init__(self, position, rayon):
        self._position = position
        self._rayon = rayon
    def get_pos_antenne(self):
        return self._position
    def get_rayon(self):
        return self._rayon
```

1. **Donner** le code qui crée et initialise deux variables `m1` et `m2` avec des instances de la classe `Maison` situées aux abscisses 1 et 3,5 (Figure 1).

On ajoute à présent une antenne ayant un rayon d'action de 1 à la position 2,5 :

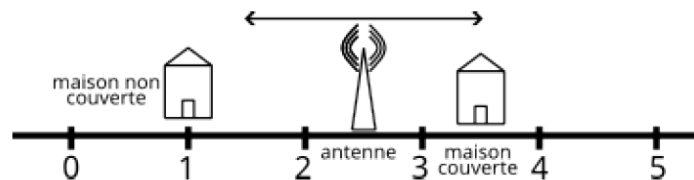


Figure 2. L'antenne placée en 2,5 et de rayon d'action 1 couvre la maison en 3,5 mais pas celle en 1

2. **Donner** le code qui crée la variable `a` correspondant à l'antenne à la position 2,5 avec le rayon d'action 1.

On souhaite modéliser une rue par une liste d'objets de type `Maison`. Cette liste sera construite à partir d'une autre liste contenant des nombres correspondant aux positions des maisons. La fonction `creation_rue` réalise ce travail. Elle prend en paramètre une liste de positions et renvoie une liste d'objets de type `Maison`.

3. **Recopier** le schéma ci-dessous et le **compléter** pour donner une représentation graphique de la situation créée par :

```
creation_rue([0, 2, 3, 4, 5, 7, 9, 10.5, 11.5])
```

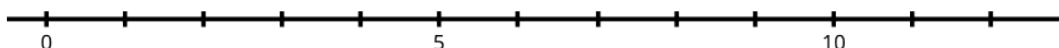


Figure 3. Axe pour représenter le problème avec 9 maisons



4. Compléter le code donné ci-dessous de la fonction `creation_rue`.

```
1 def creation_rue(pos):
2     pos.sort()
3     maisons = []
4     for p in pos:
5         m = Maison(p)
6         maisons.append(...)
7     return ...
```

Pour rappel : la commande `tab.sort()` trie la liste `tab`.

La méthode `couvre` de la classe `Antenne` prend en paramètre un objet de type `Maison` et indique par un booléen si l'antenne couvre la maison en question ou non. La méthode peut être utilisée ainsi (en supposant que les objets précédents `m1`, `m2` et `a` existent).

```
>>> a.couvre(m1) False
>>> a.couvre(m2) True
```

5. Compléter la fonction `couvre`, ci-dessous, en veillant à ne pas accéder directement aux attributs d'une maison depuis la classe `Antenne` (on pourra utiliser les méthodes `get_pos_maison`, `get_pos_antenne` et `get_rayon`).

Pour rappel : la fonction valeur absolue se nomme `abs()` en Python.

```
1 # Méthode à ajouter dans la classe Antenne
2 def couvre(self, maison):
3     # Code à compléter (éventuellement plusieurs lignes)
4     ...
```

La fonction `strategie_1` est donnée ci-dessous. L'objectif est de placer des antennes dans une rue. Elle est fournie à la société qui place les antennes. La fonction prend en paramètre une liste d'objets de type `Maison` (qu'on supposera triée par abscisse croissante) et le rayon d'action des antennes (`float`). Cette fonction renvoie une liste d'objets de type `Antenne` ayant ce rayon d'action et couvrant toutes les maisons de la rue.

```
1 def strategie_1(maisons, rayon):
2     ''' Prend en paramètre une liste de maisons et le rayon
3     d'action des antennes et renvoie une liste d'antennes
4     '''
5     antennes = [Antenne(maisons[0].get_pos_maison(), rayon)]
6     for m in maisons[1:]:
7         if not antennes[-1].couvre(m):
8             antennes.append(Antenne(m.get_pos_maison(), rayon))
9     return antennes
```

Pour rappel :

- `tab[1:]` correspond aux éléments de `tab` à partir de l'indice 1 jusqu'à la fin de la liste ;
- `tab[-1]` correspond au dernier élément de la liste `tab`.

6. Indiquer ce que renvoie cette suite d'instructions après exécution.

```
>>> maisons = creation_rue([0, 2, 3, 4, 5, 7, 9, 10.5, 11.5])
>>> antennes = strategie_1(maisons, 2)
>>> print([a.get_pos_antenne() for a in antennes])
```



3. Partie 3 : Gestion de compte (sur ordinateur) – 10 pts -

Le but de cette partie est de réaliser une classe permettant la gestion de comptes bancaires

Compte
- solde : float - nom : str - prenom : str - num_compte : int
+ __str__() : str + ajouter(montant : float) : None + debiter(montant : float) : None + get_solde() : float

Diagramme de classe

Les attributs de la classes sont :

- `solde : float` le solde du compte - lors de la création du compte le solde sera 0
- `nom : str` le nom du titulaire du compte
- `prenom : str` le prénom du titulaire du compte
- `num_compte : int` un entier identifiant du compte

Les 3 derniers paramètres devront être définis lors de l'instanciation d'un compte.

Q1. Écrire une classe `Compte` dont le diagramme est donné ci-dessus

Q2. Écrire une méthode `str()` qui permettra l'affichage du compte avec `print()`.

La méthode doit renvoyer une chaîne de caractères comme :

Nom : DURAND

Prénom : Jacques

Solde : 200

Pour insérer des retours à la ligne dans une chaîne on ajoute le caractère `"\n"`

Q3. Écrire une méthode `ajouter` qui permettra de verser de l'argent sur le compte

Q4. Écrire une méthode `debiter` qui permettra de retirer de l'argent du compte.

La méthode `débiter` doit effectuer le retrait si le solde est suffisant (supérieur au montant qu'on souhaite retirer). Sinon elle doit afficher un message et ne pas effectuer le débit.

Q5. une méthode `get_solde` qui renvoie le solde du compte.

Q6. Enfin, vous ajouterez dans votre code les instructions pour créer un compte, verser de l'argent (500), puis effectuer un débit de 200 et afficher le solde du compte.

